

张鹏 · 简历

IT Software Engineering Technical Leader | 20+ 年经验 | 合肥

☎ 18955199945 | ✉ zpkx@qq.com

个人信息

姓名	张鹏	性别	男
出生日期	1980 / 8 / 17	居住地 / 户口	合肥
工作年限	20+	婚姻状况	已婚

自我评价

我具备较强的自我管理能力，积极的学习态度和出色的动手能力。广泛的知识面和善于思考的分析能力使我能够深入探讨复杂的技术难题。在 IT 和软件行业拥有多年经验和技能积累，积累了丰富的跨领域项目经验。

在团队中，我担任技术骨干和领头人，带领团队共同完成既定目标。英语听说读写流利，善于跨部门沟通与协调。

我高度自驱，致力于持续改进。积极主动地探索和钻研前沿技术，力求在每项任务中做到精益求精。凭借自律和目标导向，确保持续交付高质量成果，同时推动团队创新与成长。

技术专长

类别	技术
AI 与智能化	LangChain, MCP, RAG, AI Agent, AI Harness (Cursor, Claude Code, Copilot)
后端开发	Java + Spring (Boot, Cloud) , Python, Node.js
前端开发	Angular, React
数据存储	SQL (Oracle, MySQL, PostgreSQL) , NoSQL (Redis, MongoDB)
架构与云	Monolithic, Microservice, Cloud Native (OpenShift, Docker, Kubernetes)
研发效能	DevSecOps (CI/CD, Jenkins, SonarQube, 安全扫描)

工作经验

思科系统（中国）研发有限公司 合肥分公司

IT Software Engineering Technical Leader | 通信 / 电信 / 网络设备 | 2008/7 - 至今

► 供应链 IT Source & Finance 团队 · Tech Lead

- 团队管理**：带领 India 和 China 跨国团队，为 Cisco 供应链交付多个云平台 IT 系统；
- 架构设计**：主导 Cloud Native 架构设计与验证，确立团队微服务与云原生技术标准；
- 技术把关**：建设前后端基础框架与示例代码，作为团队 reference implementation 提升交付一致性；
- 人员培养**：主导技术与业务培训，指导开发人员，解决系统开发、运行中的复杂问题；
- 技术决策**：参与并推动部门级技术决策与标准化，输出可复用的技术方案；
- AI 落地**：推动 AI 在团队内的落地，结合 AI Harness、Coding Agent、RAG 等能力提升研发效率与代码质量。

► 云软件团队 · Tech Lead

- 团队建设**：帮助团队从 CM 配置管理到 DevOps 转型，团队规模从 09 年的 3 人扩展至 15 人，与部门经理协同达成 Business Goal；

2. **平台架构**：主导 CI/CD 平台的架构设计、搭建与实施，持续优化构建、测试与发布流程，显著缩短交付周期；通过培训与指导赋能开发人员，提升团队整体研发效率；
3. **流程治理**：定义并监管软件开发流程；
4. **自动化运维**：以自动化方式完成应用环境的搭建与管理，协助开发团队 Debug 应用运行问题；
5. **技术前瞻**：跟踪研究新技术与行业最佳实践，并推动落地。

中国雅虎

平台技术开发部 · 高级软件工程师 | 互联网 / 电子商务 | 2006/2 – 2008/6

作为平台技术开发部的核心成员，参与中国雅虎社交产品线的开发，涉及新产品的开发，比如“站长天下”，以及开发老产品的新功能和运营需求，比如“雅虎相册”。

网迅（中国）软件有限公司 合肥分公司

QA 部 · 软件 QA 工程师 | 计算机软件 | 2004/2 – 2006/2

参与多个 WebEx Meeting 项目的测试工作，包括制定测试计划，编写测试用例，执行测试过程。同时需要与美国的 PM 和其他 site 的 dev 进行沟通和协作。

安徽百朋信息技术有限公司

技术部 · 软件工程师 | 计算机软件 | 2003/2 – 2004/2

主要从事 B/S 结构的软件开发、测试和维护。维护公司内的网络和硬件，包括服务器的维护。

项目经验

SC GenAI Splunk — Supply Chain Alert Monitoring 2026/1 – 2026/4

角色：贡献者 (Contributor) **技术栈**：Python · FastAPI · LangGraph · LangChain · Azure OpenAI · LangSmith · PyYAML · OAuth2 · Webex Adaptive Card · Snowflake · SMTP · pytest · Docker · Jenkins · Conjur

📌 项目背景

供应链 GenAI Splunk 告警监控平台（平台团队统一运维），基于 FastAPI Webhook 接收 Splunk 告警，经应用识别、模式匹配与 LangGraph AI Agent 分析后，通过 邮件 + Webex 双通道智能通知。

🎯 我的贡献

- **Webex Adaptive Card 与 KB 分析**：将 Control-M 及通用 Splunk 告警通知升级为 Webex Adaptive Card（作业详情、Oracle 异常、可折叠区块等结构化展示）；集成 Confluence Wiki 故障排查 摘要并渲染至卡片；修复 OAuth2 精确匹配 校验问题；
- **AI Monitoring Metrics 集成**：通过 CronJob 定时从 LangSmith 读取 Run/Trace 数据，写入 Snowflake 数据库，支撑平台 AI 告警链路的可观测性与指标分析；
- **Splunk 告警 Custom Fields**：在组件级 YAML 支持 custom_fields，从 Splunk result 动态提取业务字段（如 namespace、pod），贯通 Webhook → alert_analyzer → Webex / Email 全链路展示。

GenAI Self Service Hub Service 2026/2 – 2026/4

角色：贡献者 (Contributor) **技术栈**：Python 3.11+ · FastAPI · Pydantic · PyYAML · Alation REST API · boto3 · S3 · Poetry · pytest

📌 项目背景

Cisco GenAI 平台的 Self Service Hub 后端微服务，为业务团队提供知识库、文档、Agent Workflow、MCP Registry 等自助能力。本人负责其中 Semantic Layer（语义层）能力建设：对接企业数据目录 Alation，按组件 / 应用 / 表选择自动生成可供 GenAI 使用的 Semantic Model YAML，并持久化至对象存储供下游消费。

🎯 我的贡献

- **Alation 元数据集成**：实现 Alation Cloud REST Client（Token 刷新、表检索、列级元数据拉取），将 Alation 返回的表 / 列描述、数据类型、主键等信息映射为 YAML 构建所需的结构化规格；

- **Semantic YAML 生成**：基于 PyYAML 实现语义模型文档组装与校验（维度 / 事实拆分、关系与 Verified Queries 等），通过 FastAPI `**/generate` 接口按用户选择的库表一键生成 POC 对齐的 Semantic Model YAML；
- **YAML 持久化至 S3**：完成生成结果的校验与落盘，将 YAML 上传至 S3（按组件 / 应用命名与版本化管理），支撑多环境部署与下游 GenAI 流水线读取；
- **API 与质量保障**：交付 `**/generate` / `/validate` / `/save` 等 REST 端点及 Pydantic Schema；补充 `pytest` 单元测试（Alation enrichment、YAML Builder、路由层），保障元数据拉取与 YAML 生成链路的稳定性。

Cisco IT Supply Chain GenAI Platform 2025/6 – 2026/1

角色：贡献者（Contributor） **本人技术栈**：Python · Streamlit · Duo SSO · GPU（CUDA）· OpenStack · HTTPS / TLS · Poetry · CI/CD · Ansible · Linux VM **平台技术栈（团队）**：Python · FastAPI · Streamlit · LangChain · LangGraph · Azure OpenAI · RAG · MCP

📌 项目背景

团队基于 RAG + LangGraph 为 Cisco 供应链构建企业内部 AI 问答与自动化平台，集成内部多个企业系统，实现「智能问答 + 业务自动化」能力。本人作为贡献者，负责其中多项基础能力建设。

🎯 我的贡献

- **企业级 SSO 集成**：完成 Duo SSO 与 Streamlit UI 的集成，填补平台原本缺失的统一用户登录能力，提升企业级安全合规；
- **GPU 算力启用**：在生产 VM 上完成 Nvidia GPU 驱动安装与配置，为平台 Embedding 向量化加速提供算力底座，显著提升知识库检索性能；
- **多环境 HTTPS / 证书部署**：在 OpenStack 平台上为 Dev / Stage / Prod 三套运行环境完成 HTTPS 与 TLS 证书配置，统一传输层加密标准，满足企业安全合规要求；
- **依赖管理与 CI/CD 建设**：将项目依赖管理从 pip 迁移至 Poetry（确保可复现构建与依赖锁定），并完成 CI/CD Pipeline 搭建，实现自动化构建、测试与部署，缩短交付周期；
- **Ansible 自动化基础搭建**：在 Dev 主机上完成 Ansible 安装与配置，建立面向多 VM 的自动化运维基础能力，为团队后续通过 Playbook 实现批量化、可复用的自动化运维任务铺路。

API Maestro — Cisco IT Hackathon 2025 2025/5 – 2025/6

角色：独立开发者（Solo, Cisco IT Hackathon 2025） **技术栈**：Python 3.12 · LangChain · LangGraph · FastMCP · Streamlit · FastAPI · OpenAPI / Swagger · BridgeIT · Docker Compose · PyYAML · Requests · ReAct Agent

📌 项目背景

API Maestro 是本人 Solo 独立完成的 Cisco IT Hackathon 2025 参赛作品，面向「用自然语言驱动任意 OpenAPI 应用」的场景：系统自动解析 OpenAPI/Swagger 规范，理解端点、参数与 workflow，将用户的口语化指令（如「查询最近 10 条订单」「向所有活跃用户发送通知」）翻译为精准的 HTTP API 调用并返回可读结果，让非技术用户也能零代码操作 API 驱动的业务系统。

🎯 责任描述

- **从 0 到 1 独立交付**：独立完成架构设计、核心编码、Demo 与容器化部署，在约 2 个月内交付可演示的完整作品；
- **Agent 核心架构**：基于 LangGraph ReAct Agent 构建 NL → API 编排链路，动态将 OpenAPI 端点映射为 LangChain Tools，结合 BridgeIT 内部 LLM 完成参数抽取与 API 执行；
- **OpenAPI 动态学习**：实现 OpenAPI 规范解析与端点工具化，支持运行时加载/切换 API Spec，无需手工编写每个接口的调用代码；
- **MCP Server 演进**：将原有 Agent 能力重构为基于 FastMCP 的 Model Context Protocol 服务，提供 `load_spec`、`execute_api`、`process_request` 等标准化 Tool，支持 Cursor / IDE 等 MCP 客户端接入；
- **多通道接入**：交付 Streamlit 对话式 Web UI（上传 Spec、会话管理、流式交互）与 FastAPI HTTP Bridge，并配套 Mock API Server 用于演示与联调；
- **工程化交付**：完成 Docker Compose 一键部署、环境变量安全管理（dotenv）、CLI 入口（`api-maestro-server` / `api-maestro-ui`）及 MCP stdio 测试脚本，支撑 Hackathon 现场 Demo 与后续扩展。

Supply Chain Finance Miscellaneous Tools 2023/9 – 至今

技术栈：Java 21 · Spring Boot 4.0 · Spring Security · OAuth2 · Cisco LDAP · OpenAPI v3 · Angular 21 · Nx · Cisco Design System · Gradle (Kotlin DSL) · OpenRewrite · Docker (Chainguard) · Jenkins · SonarQube · Spinnaker · Conjur

📌 项目描述

面向供应链财务团队的轻量级工具集平台，承载多个高频内部业务工具（Item Cost Download、Cost Tag Upload 等），覆盖 Excel/CSV 批量上传、成本数据下载、邮件通知等场景。通过 Cisco LDAP 实现统一身份接入与基于角色的细粒度访问控制（General User / Cost Manager 等）。

🎯 责任描述

- **Monolithic 一体化架构：**Angular SPA + Spring Boot 同仓共建，Gradle (Kotlin DSL) 统一编排前后端构建，输出单一可执行 Jar；
- **企业级安全 + Cisco Design System：**基于 Spring Security + OAuth2 + Cisco LDAP 实现 SSO 与 RBAC 双重校验；前端对齐 Cisco 设计规范并交付通用组件（文件上传器、Session 过期弹窗等）；
- **API-First：**维护 OpenAPI v3 规范并落地代码自动生成（Spring 接口 + TypeScript SDK），结构性消除前后端契约漂移；
- **持续技术演进：**通过 OpenRewrite 主导 Spring Boot 2 → 3.5 → 4.0、Angular → 21 等多次跨大版本升级，沉淀可复用升级范式；
- **DevSecOps：**搭建 Jenkins + SonarQube Quality Gate + SAST + Spinnaker 自动化交付链路；镜像迁移至 Chainguard 安全基础镜像；密钥托管接入 Conjur；维护 8 套环境配置并主导生产凭证轮换与漏洞修复。

CART-Kafka — CBM 数据接入微服务 2024/7 – 至今

技术栈：Java 21 · Spring Boot 4.0 · Spring Kafka · Spring Data JPA · Oracle · OAuth2 · Conjur · Maven · OpenRewrite · Docker · Jenkins (Golden Pipeline) · SonarQube · Spinnaker · OpenShift

📌 项目描述

CART 项目的上游数据接入微服务，独立于 CART 主应用部署。负责消费 Oracle Cloud 的 CBM (Component Business Model) Kafka 数据流，将组件业务数据近实时复制到企业 ESM 系统的 XXCART schema 接口表中（GL、AP、PO、RCV、Material Transactions、XLA 等十余张核心财务表），为 CART 的成本分析与对账提供原始数据底座。

🎯 责任描述

- **架构与主导：**作为 Tech Lead 主导项目从 0 到 1 的架构设计与搭建，确立“独立微服务 + 双 Oracle 数据源 (ESM / CG1)”的分层设计，将数据接入与业务计算解耦；
- **核心代码贡献：**完成 Kafka Consumer 框架、双数据源自动配置、CBM 实体模型与持久化层、处理日志等核心模块的设计与编码；
- **批量插入性能优化：**通过 Hibernate jdbc.batch_size + order_inserts / order_updates 配合按 batch_size 切片调用 saveAll 的写入策略，让 Hibernate Batch 真正落地（避免逐条 flush），显著提升大批量 CBM 数据写入 Oracle 的吞吐能力；
- **持续技术演进：**通过 OpenRewrite 主导 Spring Boot 2.x → 3.2 → 3.5 → 4.0、Java → 21 多轮升级（配合 Confluent Kafka 7.6 适配），沉淀可复用升级范式；
- **DevSecOps：**完成 Cert Auth → OAuth2 迁移与 Conjur 密钥托管接入；落地 SonarQube Quality Gate + SAST + Dependabot + PMD/CPD + JaCoCo 多重质量门禁；构建迁移至 Cisco IT Golden Pipeline，统一构建、安全扫描、镜像发布与 Spinnaker 部署；
- **多环境与团队赋能：**维护 9 套环境配置，主导生产证书轮换与漏洞修复（Qwint / Tomcat CVE 等）；作为 PR 主 Reviewer 把关代码质量与架构一致性，指导团队成员完成大量 feature 开发。

CART — Cost Analysis and Reconciliation Tool 2020/9 – 至今

技术栈：Java 21 · Spring Boot 4.0 · Spring Security · OAuth2 · Cisco LDAP · Spring Data JPA · Oracle · Redis · OpenAPI v3 · Apache POI · AWS S3 / Cisco COS · Micrometer Tracing · OWASP ESAPI · Gradle ·

📌 项目描述

供应链财务核心业务系统，自动化日结流程中的**成本差异分析与子账 (CG1) / 总账 (CFN) / Cloud (CBM)** 三源对账，并将分配 / 未分配数据推送至 **DFA / ICPM** 管理报表系统。前后端独立部署 (**cart-api** + **cart-ui** 双 Repo)，后端 22 个 Controller / 43 个 Service，前端 17 个业务模块，是部门内体量较大的真实业务系统。

🎯 责任描述

- **全栈脚手架与架构**：基于此前项目成熟经验，主导搭建前后端分离的全栈脚手架，并演进为多团队共建的参考实现；
- **企业级安全**：基于 **Spring Security + OAuth2 Client + Resource Server + Cisco LDAP** 实现 SSO、RBAC 与服务间鉴权，引入 **OWASP ESAPI** 进行输入安全防护；
- **Object Storage 设计与实现**：完成 **Cisco COS / AWS S3** 集成的设计与编码，统一抽象对象存储能力（含 bucket lifecycle policy 管理），覆盖大文件上传、归档与下载场景；
- **持续技术演进**：通过 **OpenRewrite** 主导 **Spring Boot** 升级至 4.0、**Java** → 21、**Angular** 升级至 21 等多次跨大版本演进；接入 **Micrometer Tracing + Brave** 分布式追踪；
- **DevSecOps 与基础设施**：负责 **Jenkins + SonarQube Quality Gate + SAST + PMD + JaCoCo + Spinnaker** 自动化交付链路与 **OpenShift** 多 DC 运行环境的设计搭建；**Conjur** 密钥托管；维护多套环境配置并主导生产凭证轮换与漏洞修复。

Earlier Projects (2003 — 至今)

时间	项目	角色与亮点
2018/7 - 至今	RCE — Royalty Calculation Engine	Tech Lead (9 人跨国团队)；Cisco 专利税核心业务系统 (30+ 模块)；Cloud Native + API-First；部门首个 Blue-Green 零停机部署；Nx Monorepo 与 @scf/common-lib 共享库；OpenRewrite 演进至 Spring Boot 4 / Java 21 / Angular 21
2014/11 - 2018/1	Self Service Model	优化构建 / 部署流程，实现 Build-Once / Deploy-Many 与 Dev / QA Self Service 上线；开发 Continuous Delivery Portal；流程制定、架构选型、POC 与团队培训
2013/11 - 2014/10	Continuous Delivery Transformation	Cisco IT CDT 统一交付平台；主导 Git / Jenkins / Artifactory 迁移至新平台；uDeploy / uRelease 可行性研究与美印平台团队协作
2012/7 - 2013/10	Data Center Migration	WebEx DataCenter → Cisco DataCenter 应用迁移；赴美参与架构讨论与 POC；编写 WLST 自动化脚本提升环境搭建效率与可重用性
2011/7 - 2012/6	Git Migration	CVS → Git 迁移；完成 POC 与技术方案；英语培训 Dev 团队 Git 技能；带领 DevOps 成员按计划完成迁移
2009/9 - 2010/10	Continuous Integration	重构 CruiseControl 脚本；主导从 CruiseControl 迁移至 Hudson (Jenkins)
2008/8 - 2009/8	WebEx Elvis	维护内部生命周期管理系统 (PHP + MySQL)，完成 Bug 修复与功能开发
2008/7 - 2009/8	Myify	PHP / Zend / Dojo 任务跟踪 Web 应用主开发，完成 90%+ 代码
2007/10 - 2008/5	Y!China NBS Platform	雅虎站长天下建站平台前端主开发，主导 symfony 框架落地与后台管理模块
2007/5 - 2007/7	Y!China Blog UGC	雅虎博客聚合前端架构设计与开发 (PHP + PostgreSQL)
2006/2 - 2007/5	Y!China Photo Service	雅虎相册前端主开发，配合运营将 PV 从 <100 万提升至 2000 万+
2003/10 - 2004/2	国家外资年检系统	商务部联合 7 部委的 .NET 系统

业余项目

RCE UI 重写为 React 2026/4 - 2026/5

技术栈：React 19 · TypeScript 6 · Vite 8 · Vitest · Playwright · React Router 7 · TanStack Query v5 (含持久化) · AG Grid Enterprise · Orval (OpenAPI → React Query hooks) · OIDC Client / React OIDC Context · Cisco Atmosphere Elements React 4.0 · Harbor · Tailwind CSS · pnpm Monorepo · Docker · AI Harness (Cursor / Claude Code / Copilot)

📌 背景与动机

作为一项个人主导的探索性 Initiative，目标是**多重价值并举**：

- **验证 Vibe Coding 模式与 AI Harness 实践**：以 AI Coding Agent 为主要“动手”角色、人作为架构指挥与质量把关者，验证这种新型人机协作范式（Vibe Coding）在跨技术栈整体重写这一高复杂度场景下的可行性与效率边界；同时**在实践中实践 AI Harness 方法论**（Prompt 设计、上下文管理、Skill 编排、人机分工、质量校验），积累可被团队复用的工程化经验；
- **UI/UX 对齐最新标准**：原 RCE 前端基于早期 Angular 实现，UI/UX 已不符合最新 Cisco Design System 规范，借此机会完成视觉与交互的现代化升级（升级至 Atmosphere v4）；
- **代码质量提升**：以 React 19 + 严格 TypeScript 为底座，重构原有耦合较重的代码结构，建立更清晰的模块划分与组件边界；
- **性能优化**：利用 React 19 新特性（并发渲染、自动批处理等）显著优化首屏加载与运行时性能。

🎯 价值与产出

- **全量重写**：仅用约 2 个月独立完成原本需团队级投入的全量业务页面重写（**22 个业务页面**），验证 Vibe Coding 在端到端架构迁移中的可行性；
- **可复用脚手架与方法论**：沉淀 **pnpm Monorepo + React 19 + Vite + Orval + TanStack Query** 现代前端脚手架与参考实现；输出 **React vs Angular 差异分析文档、分支策略、发布 checklist** 等工程化文档，为团队后续 React 选型项目提供模板；
- **企业级能力适配**：完成 **OIDC SSO、AG Grid Enterprise、多环境运行时配置注入、Docker 容器化部署、Playwright E2E 测试** 等企业级能力适配，让原型级别的重写直接达到生产可用水平；
- **AI Harness 工程化**：综合运用 **Superpowers 插件与自建 Skills**对 AI Coding Agent 进行 Harness（约束与赋能），形成可复用的 Agent 工作流（涵盖 TDD、系统化调试、代码审查、计划执行等模式），显著提升 AI 在复杂任务中的可控性与产出质量。

Playwright Test Automation 2025 - 2026

技术栈：Playwright · TypeScript · Page Object Model · monocart-reporter · oracledb · TA JWT (RSA) · DUO OIDC · OAuth2 JWT · Express · Multer · Docker · Jenkins · Conjur · Node.js

📌 背景与动机

团队多个项目（RCE、CART、Supply Chain Finance Tools 等）长期缺少 E2E 自动化测试，质量保障高度依赖手工回归，导致发布节奏受限、回归成本高。发起此 Initiative，目标是为团队引入一套**现代化、可复用、跨项目通用**的端到端自动化测试体系。

🎯 价值与产出

- **跨项目测试框架**：基于 Playwright + TypeScript + Page Object Model 从 0 搭建跨应用通用的自动化测试框架，覆盖 **CART (E2E + API)、RCE (E2E / Regression / Smoke)** 多套测试矩阵，并沉淀 **AG Grid 自动化工具、Oracle DB 直连数据校验、邮件通知** 等通用能力；
- **企业级认证适配**：设计并实现 **TA JWT (RSA 签名) 测试态认证方案**，与生产应用配套联动（私钥经 Conjur 托管、CI 中安全注入），彻底解决企业 SSO 在自动化场景下无法登录的难题，让测试稳定无人值守运行；
- **CI/CD 集成与容器化执行**：自构建并维护专用 **Playwright Docker 镜像**（含 Oracle Instant Client 等依赖）保证环境一致性，通过 **Jenkins Pipeline + 矩阵参数（应用 / 环境 / Project / Tag）** 调度执行，每次 PR / 发布自动运行回归套件，显著降低回归成本、加快发布节奏；
- **企业级测试报告服务**：自建 **Test Report Server (Express + Multer)**，**DUO OIDC 保护报告浏览、OAuth2 JWT 保护上传接口**，**Docker 化部署**，集中托管历史报告，便于团队追踪趋势与定位回归问题；
- **跨项目复用与标准化**：作为可复用模板被团队多个项目采纳，统一了团队的 E2E 测试标准与最佳实践。

Video Downloader 2025/6 - 2025/7

技术栈：Python 3.12 · yt-dlp · PyYAML · Docker · Docker Compose · GitHub Actions · GHCR · Bash **开源仓库**：github.com/zpkx/video-downloader（个人 GitHub 公开）

📌 背景与动机

个人业余项目，**源码已在个人 GitHub 开源公开**，解决多平台视频批量下载与归档的痛点：手工维护 URL 列表、按类别分目录保存、避免触发限流，并希望在 NAS / 容器环境中**改配置即自动下载**，无需反复手工执行命令。

🎯 价值与产出

- 核心下载引擎：基于 yt-dlp 封装 `VideoDownloader` 类，支持单 URL、多 URL、按类别批量下载，以及 `info-only / dry-run` 等非破坏性操作模式；
- YAML 驱动配置：用 `urls.yaml` 定义分类目录、画质、延迟区间与 URL 列表，用 `config.yaml` 集中管理 yt-dlp 选项（字幕、合并格式、重试策略等），支持 `config/` 目录自动探测加载；
- Docker 化与自动化：提供 `Dockerfile + docker-compose`（`watcher / oneshot / manual` 多服务模式），实现 `urls.yaml` 变更即触发下载 的文件监控；支持可配置 `UID/GID` 避免挂载卷权限问题；
- 运维与发布：配套 `docker-run.sh` 等辅助脚本、分级日志（`logs/video_download.log`）、Cookie 自动探测；通过 GitHub Actions 构建并发布镜像至 GHCR（`ghcr.io/zpkx/video-downloader`），便于家庭 NAS 长期拉取运行。

教育经历

时间	学校	专业	学历
1999/9 – 2003/7	合肥工业大学	计算机应用	本科
1996/9 – 1999/7	安徽省安庆市二中	—	高中

证书

时间	证书	备注
2020/5	AWS Certified Solutions Architect — Associate	977
2015/1	Certified Scrum Master	—
2014/6	Oracle Certified Associate, Oracle WebLogic Server 11g System Administrator	—
2003/1	大学英语四级	

语言能力

语言	水平
英语	听说读写（熟练）